

Logger - How it works

Home notes + DESIGN.md + source code | compact print

In one line - You call `logger.error('msg')` -> message walks a chain of level handlers -> subscribed outputs print/write it.

Story - how this LLD works

Like Log4j in miniature. Logger is global (Singleton). Handlers form a chain: Debug -> Info -> Warn -> Error -> Fatal. When Error matches, it notifies appenders (console, file). Each appender formats the text (plain or JSON) then writes.

Who does what

Logger: Your API: `info()`, `error()`, ... Builds `LogMessage` and starts the chain.

LogHandler chain: Each handler checks 'is this my level?' else pass to next.

LogAppender: Where it goes: console or file.

LogFormatter: How it looks: plain text or JSON-like.

LogMessage: level + text + timestamp.

Main pieces

Logger

- `getInstance`
- `info/error/...`

Handler chain

- Debug->...->Fatal
- exact level match

Appenders

- Console
- File + lock

Formatter

- Plain
- JSON-ish

Step-by-step at runtime

1. `logger.error('disk full')` creates `LogMessage(ERROR, ...)`.
2. `DebugHandler`: not my level -> pass.
3. `InfoHandler`, `WarnHandler`: pass.
4. `ErrorHandler`: match! notify all subscribed appenders.
5. `ConsoleAppender` formats and prints; `FileAppender` locks and appends to file.
6. Done. Fatal would travel one step further if level were FATAL.

Why this design (plain words)

Why chain? - Different levels can have different outputs (info only console, error also file).

Why Observer (subscribe)? - Add a new destination without changing Logger.

Why Strategy formatter? - Same file appender can switch plain vs JSON.

Remember

- This demo matches EXACT level, not 'Error and above'.
- TRACE exists in enum but is not wired in the chain.
- File appender uses a lock for concurrent writes.

OOP in this project

Encapsulation - Callers never touch the chain - only Logger methods.

Abstraction - `append(message)` / `format(message)` hide destinations.

Inheritance - Each level has its handler class.

Polymorphism - `handle()` behaves differently per handler.

DESIGN.md - full file

Complete markdown from lld/logger/DESIGN.md (Q&A for learning).

Full DESIGN.md (same file as in this folder) - DESIGN.md

Designing a Logger Framework - LLD Q&A

Run: `python -m lld.logger.main`

Patterns: Singleton · Chain of Responsibility · Observer · Strategy

Q1. What problem are we solving?

A. Design a configurable logging framework (Log4j/SLF4J style) that:

Logs at multiple severity levels (DEBUG -> FATAL)

Routes each level to subscribed appenders (console, file)

Formats messages independently (plain text / JSON)

Exposes a simple global API: `logger.info(...)`, `logger.error(...)`

Q2. What are the core entities?

- Entity - Responsibility
- Logger - Singleton facade - builds `LogMessage`, dispatches to chain
- `LogHandler` - Chain node per level; holds subscribed appenders
- `LogAppender` - Output sink (Console / File)
- `LogFormatter` - Message format strategy (Plain / JSON)
- `LogMessage` - Level + text + timestamp
- `LogHandlerConfiguration` - Wires chain + appender subscriptions

Q3. How is the package structured?

```
logger/
|-- logger.py          # Singleton API
|-- log_handler_configuration.py # Wire-up
|-- model/             # LogMessage
|-- handlers/          # Debug -> Info -> Warn -> Error -> Fatal
|-- appenders/         # ConsoleAppender, FileAppender
|-- formatter/         # PlainText, Json
|-- enums/             # LogLevel
```

Q4. Why Chain of Responsibility?

A. Handlers are linked: DEBUG -> INFO -> WARN -> ERROR -> FATAL. Each tries to handle its level; otherwise passes downstream.

```
def handle(self, message):
    if self.can_handle(message.level):
        self.notify_observers(message)
    elif self.next is not None:
        self.next.handle(message)
```

Note: Exact level match (not "this level and above").

Q5. Why Observer on handlers?

A. Appenders subscribe to handlers. When a handler matches, it notifies all subscribed appenders.

Demo: INFO -> console only; ERROR -> console and file.

Adding a new destination = new appender + subscribe - no change to Logger.

Q6. Why Strategy for formatters?

A. Same appender can use different formats:

```
ConsoleAppender(PlainTextFormatter())
FileAppender(JsonFormatter())
```

Q7. Why Singleton for Logger?

A. One global logger instance for the app - typical logging API.

```
Logger.get_instance().error("something failed")
```

Q8. What is the log flow?

```
logger.error("msg")
-> LogMessage(ERROR, msg, timestamp)
-> DebugHandler -> pass -> ... -> ErrorHandler matches
-> notify appenders -> formatter.format() -> print / write
```

Q9. How do you extend this design?

- Add... - How
- Syslog / Kafka appender - Implement `LogAppender`
- Async logging - Queue + worker thread in `FileAppender`
- Level threshold - Change `can_handle` to `level >= threshold`
- Config file - Externalize `LogHandlerConfiguration`

Q10. Common interview follow-ups

Q. Chain vs single handler with filter?

Chain makes per-level appender routing natural (ERROR to file, INFO to console). Threshold style is simpler for "log everything $\geq$ WARN".

Q. FileAppender thread safety?

Uses threading.Lock around writes.

Q. TRACE level?

Exists in enum but no TraceHandler wired in this demo.

Source code - lld/logger/

17 Python files below (compact font to save pages). Read with the notes: find the class name from the cast, then open that file here.

- Tip: start at main.py, then service/, then model/ + strategy/.

FILE: main.py (33 lines)

```
1| from .appenders.console_appender import ConsoleAppender
2| from .appenders.file_appender import FileAppender
3| from .enums.log_level import LogLevel
4| from .formatter.plain_text_formatter import PlainTextFormatter
5| from .log_handler_configuration import LogHandlerConfiguration
6| from .logger import Logger
7|
8|
9| def main() -> None:
10|     logger = Logger.get_instance()
11|
12|     LogHandlerConfiguration.add_appender_for_level(
13|         LogLevel.INFO,
14|         ConsoleAppender(PlainTextFormatter()),
15|     )
16|
17|     LogHandlerConfiguration.add_appender_for_level(
18|         LogLevel.ERROR,
19|         ConsoleAppender(PlainTextFormatter()),
20|     )
21|
22|     LogHandlerConfiguration.add_appender_for_level(
23|         LogLevel.ERROR,
24|         FileAppender(PlainTextFormatter(), "logs.txt"),
25|     )
26|
27|     # Usage
28|     logger.info("This is some key information") # CONSOLE
29|     logger.error("Oh no! there's an error") # CONSOLE + FILE
30|
31|
32| if __name__ == "__main__":
33|     main()
```

FILE: model/log_message.py (10 lines)

```
1| from dataclasses import dataclass
2|
3| from ..enums.log_level import LogLevel
4|
5|
6| @dataclass
7| class LogMessage:
8|     level: LogLevel
9|     message: str
10|     timestamp: int
```

FILE: appenders/console_appender.py (11 lines)

```
1| from ..formatter.log_formatter import LogFormatter
2| from ..model.log_message import LogMessage
3| from .log_appender import LogAppender
4|
5|
6| class ConsoleAppender(LogAppender):
7|     def __init__(self, formatter: LogFormatter):
8|         self._formatter = formatter
9|
10|     def append(self, message: LogMessage) -> None:
11|         print(self._formatter.format(message))
```

FILE: appenders/file_appender.py (41 lines)

```
1| import threading
2| import traceback
3|
4| from ..formatter.log_formatter import LogFormatter
5| from ..model.log_message import LogMessage
6| from .log_appender import LogAppender
7|
8|
9| class FileAppender(LogAppender):
10|     def __init__(self, formatter: LogFormatter, file_name: str):
11|         self._formatter = formatter
12|         self._lock = threading.Lock()
13|         try:
14|             self._file = open(file_name, "a", encoding="utf-8")
15|         except OSError as e:
16|             raise RuntimeError("Failed to open log file") from e
17|
18|     # t1, t2, t3,
19|     def append(self, message: LogMessage) -> None:
20|         # t4, t5
21|
22|         # blocking queue -> worker threads
23|         with self._lock:
24|             try:
25|                 self._file.write(self._formatter.format(message))
26|                 self._file.write("\n")
27|                 self._file.flush() # flush can be batched or delayed
28|             except OSError:
29|                 traceback.print_exc()
30|
31|     # t1 t2 t3
32|
```

```
33| # blocking queue of cap = 3
34| # 3 worker threads
35|
36| def close(self) -> None:
37|     with self._lock:
38|         try:
39|             self._file.close()
40|         except OSError:
41|             traceback.print_exc()
```

FILE: appenders/log_appender.py (9 lines)

```
1| from abc import ABC, abstractmethod
2|
3| from ..model.log_message import LogMessage
4|
5|
6| class LogAppender(ABC):
7|     @abstractmethod
8|     def append(self, message: LogMessage) -> None:
9|         pass
```

FILE: enums/log_level.py (10 lines)

```
1| from enum import Enum
2|
3|
4| class LogLevel(Enum):
5|     TRACE = "TRACE"
6|     DEBUG = "DEBUG"
7|     INFO = "INFO"
8|     WARN = "WARN"
9|     ERROR = "ERROR"
10|    FATAL = "FATAL"
```

FILE: formatter/json_formatter.py (18 lines)

```
1| from datetime import datetime
2|
3| from ..model.log_message import LogMessage
4| from .log_formatter import LogFormatter
5|
6| _FORMATTER = "%Y-%m-%d %H:%M:%S"
7|
8|
9| class JsonFormatter(LogFormatter):
10|     def format(self, message: LogMessage) -> str:
11|         formatted_time = datetime.fromtimestamp(message.timestamp / 1000).strftime(
12|             _FORMATTER
13|         )
14|
15|         return (
16|             f'{{"timestamp": {formatted_time}, "level": "{message.level.value}", '
17|             f'"message": "{message.message}"}}'
18|         )
```

FILE: formatter/log_formatter.py (9 lines)

```
1| from abc import ABC, abstractmethod
2|
3| from ..model.log_message import LogMessage
4|
5|
6| class LogFormatter(ABC):
7|     @abstractmethod
8|     def format(self, message: LogMessage) -> str:
9|         pass
```

FILE: formatter/plain_text_formatter.py (15 lines)

```
1| from datetime import datetime
2|
3| from ..model.log_message import LogMessage
4| from .log_formatter import LogFormatter
5|
6| _FORMATTER = "%Y-%m-%d %H:%M:%S"
7|
8|
9| class PlainTextFormatter(LogFormatter):
10|     def format(self, message: LogMessage) -> str:
11|         formatted_time = datetime.fromtimestamp(message.timestamp / 1000).strftime(
12|             _FORMATTER
13|         )
14|
15|         return f'[{formatted_time} {message.level.value}] - {message.message}'
```

FILE: handlers/debug_handler.py (7 lines)

```
1| from ..enums.log_level import LogLevel
2| from .log_handler import LogHandler
3|
4|
5| class DebugHandler(LogHandler):
6|     def can_handle(self, level: LogLevel) -> bool:
7|         return level == LogLevel.DEBUG
```

FILE: handlers/error_handler.py (7 lines)

(cont.) handlers/error_handler.py

```

1| from ..enums.log_level import LogLevel
2| from .log_handler import LogHandler
3|
4|
5| class ErrorHandler(LogHandler):
6|     def can_handle(self, level: LogLevel) -> bool:
7|         return level == LogLevel.ERROR

```

FILE: handlers/fatal_handler.py (7 lines)

```

1| from ..enums.log_level import LogLevel
2| from .log_handler import LogHandler
3|
4|
5| class FatalHandler(LogHandler):
6|     def can_handle(self, level: LogLevel) -> bool:
7|         return level == LogLevel.FATAL

```

FILE: handlers/info_handler.py (7 lines)

```

1| from ..enums.log_level import LogLevel
2| from .log_handler import LogHandler
3|
4|
5| class InfoHandler(LogHandler):
6|     def can_handle(self, level: LogLevel) -> bool:
7|         return level == LogLevel.INFO

```

FILE: handlers/log_handler.py (33 lines)

```

1| from abc import ABC, abstractmethod
2|
3| from ..appenders.log_appender import LogAppender
4| from ..enums.log_level import LogLevel
5| from ..model.log_message import LogMessage
6|
7|
8| class LogHandler(ABC):
9|     def __init__(self):
10|         self.next: LogHandler | None = None
11|         self._appenders: list[LogAppender] = []
12|
13|     def subscribe(self, observer: LogAppender) -> None:
14|         self._appenders.append(observer)
15|
16|     def notify_observers(self, message: LogMessage) -> None:
17|         for appender in self._appenders:
18|             appender.append(message)
19|
20|     def handle(self, message: LogMessage) -> None:
21|         if self.can_handle(message.level):
22|             self.notify_observers(message)
23|         elif self.next is not None:
24|             self.next.handle(message)
25|
26|     @abstractmethod
27|     def can_handle(self, level: LogLevel) -> bool:
28|         pass
29|
30|
31| # [console, file]
32|
33| # removes the file

```

FILE: handlers/warn_handler.py (7 lines)

```

1| from ..enums.log_level import LogLevel
2| from .log_handler import LogHandler
3|
4|
5| class WarnHandler(LogHandler):
6|     def can_handle(self, level: LogLevel) -> bool:
7|         return level == LogLevel.WARN

```

FILE: log_handler_configuration.py (38 lines)

```

1| from ..appenders.log_appender import LogAppender
2| from ..enums.log_level import LogLevel
3| from .handlers.debug_handler import DebugHandler
4| from .handlers.error_handler import ErrorHandler
5| from .handlers.fatal_handler import FatalHandler
6| from .handlers.info_handler import InfoHandler
7| from .handlers.log_handler import LogHandler
8| from .handlers.warn_handler import WarnHandler
9|
10|
11| class LogHandlerConfiguration:
12|     _debug = DebugHandler()
13|     _info = InfoHandler()
14|     _warn = WarnHandler()
15|     _error = ErrorHandler()
16|     _fatal = FatalHandler()
17|
18|     @classmethod
19|     def build(cls) -> LogHandler:
20|         cls._debug.next = cls._info
21|         cls._info.next = cls._warn
22|         cls._warn.next = cls._error
23|         cls._error.next = cls._fatal
24|
25|         return cls._debug
26|

```

```
27| @classmethod
28| def add_appender_for_level(cls, level: LogLevel, appender: LogAppender) -> None:
29|     if level == LogLevel.DEBUG:
30|         cls._debug.subscribe(appender)
31|     elif level == LogLevel.INFO:
32|         cls._info.subscribe(appender)
33|     elif level == LogLevel.WARN:
34|         cls._warn.subscribe(appender)
35|     elif level == LogLevel.ERROR:
36|         cls._error.subscribe(appender)
37|     elif level == LogLevel.FATAL:
38|         cls._fatal.subscribe(appender)
```

FILE: logger.py (37 lines)

```
1| import time
2|
3| from .enums.log_level import LogLevel
4| from .log_handler_configuration import LogHandlerConfiguration
5| from .model.log_message import LogMessage
6|
7|
8| class Logger:
9|     _instance: "Logger | None" = None
10|
11|     def __init__(self):
12|         self._handler_chain = LogHandlerConfiguration.build()
13|
14|     @classmethod
15|     def get_instance(cls) -> "Logger":
16|         if cls._instance is None:
17|             cls._instance = Logger()
18|         return cls._instance
19|
20|     def log(self, level: LogLevel, message: str) -> None:
21|         msg = LogMessage(level, message, int(time.time() * 1000))
22|         self._handler_chain.handle(msg)
23|
24|     def debug(self, msg: str) -> None:
25|         self.log(LogLevel.DEBUG, msg)
26|
27|     def info(self, msg: str) -> None:
28|         self.log(LogLevel.INFO, msg)
29|
30|     def warn(self, msg: str) -> None:
31|         self.log(LogLevel.WARN, msg)
32|
33|     def error(self, msg: str) -> None:
34|         self.log(LogLevel.ERROR, msg)
35|
36|     def fatal(self, msg: str) -> None:
37|         self.log(LogLevel.FATAL, msg)
```